

Отчёт по исследованию строковых сортировок

SET 9, задача A1

ФИО: Субботин Лев Дмитриевич

Группа: БПИ244

Дата: 24.05.2026

1. Постановка задачи

Стандартные алгоритмы сортировки, основанные на сравнениях, имеют асимптотику $\Omega(n \cdot \log n)$ по числу сравнений. Однако в случае строк это сравнение само нелинейно — оно занимает до $O(L)$ посимвольных операций, где L — длина строк. Поэтому полезно рассматривать сложность по **посимвольным сравнениям**, а не по сравнениям-целым.

Цель работы — сравнить «обычные» сортировки массивов строк с алгоритмами, специально адаптированными к строкам:

- тернарный **String Quicksort** (Bentley–Sedgewick);
- **LCP-Mergesort** — слияние, использующее предвычисленные длины наибольших общих префиксов, хранимые **вместе со строкой**;
- **MSD Radix Sort** (без переключения);
- **MSD Radix Sort с переключением на String Quicksort** на подмассивах меньше мощности алфавита (74).

2. Инфраструктура

2.1 StringGenerator

Генератор (`string_generator.h`) поддерживает:

- алфавит из 74 символов (`A..Z`, `a..z`, `0..9`, `!@#%:;^&*()-.`);
- длины строк в диапазоне `[10, 200]`;
- три типа массивов:
 - `generateRandom` — полностью случайные;
 - `generateReverseSorted` — отсортированные в обратном порядке;
 - `generateAlmostSorted` — отсортированные с ≈ 30 случайными обменами;
- отдельно `generateWithCommonPrefix` для массивов с заранее заданным общим префиксом.

Для воспроизводимости генератор инициализируется фиксированным `seed`-ом (`StringGenerator gen(42)`). Все эксперименты используют **базовый массив длины 3000**, из которого берутся префиксы нужного размера — это гарантирует, что массив размера 200 является префиксом массива размера 300 и т.д.

2.2 StringSortTester

Класс (`string_sort_tester.h`) выполняет замеры по двум метрикам:

- **время** через `std::chrono::high_resolution_clock` (миллисекунды);
- **число посимвольных сравнений** — счётчик `int64_t`, передаваемый по ссылке в каждый алгоритм.

Каждый замер выполняется на копии массива (чтобы измерение времени и сравнений было корректным относительно одинаковых входов). Для каждой пары (размер, тип) проводится 5 повторов, результаты усредняются. Запись результатов — в `result.csv`.

2.3 Правила подсчёта сравнений

- **Standard sort / stable_sort** — компаратор инкрементирует счётчик ровно один раз за каждую пару `(a[i], b[i])`.
- **String Quicksort** — одно сравнение за каждое обращение `charAt(arr[i], d)`.
- **LCP-Mergesort** — одно сравнение за фактическое посимвольное сравнение в `lcpCompareFrom`. Случаи, когда сравнение пропускается за счёт правил наследования LCP, **не считаются** — именно в этом проявляется выигрыш алгоритма.
- **MSD Radix** — одно сравнение за каждое обращение к символу при заполнении массива `count` (фаза подсчёта).

3. Особенности реализаций

3.1 LCP-Mergesort — ключевая часть

Реализация (`merge_sort_lcp.h`) следует спецификации: каждая строка хранится в структуре `StrEntry { string str; int lcp; }`, где `lcp` — длина наибольшего общего префикса с **предыдущей** строкой в отсортированном фрагменте. Это позволяет при слиянии **не пересчитывать LCP заново**, а использовать правило наследования:

Пусть при слиянии L и R известно $h = \text{LCP}(L[i], R[j])$ и при этом $L[i] < R[j]$ (либо наоборот). После того, как мы выводим $L[i]$ и переходим к $L[i + 1]$, имеем $m = L[i + 1].\text{lcp} = \text{LCP}(L[i], L[i + 1])$. Тогда:

- если $m > h$: $L[i + 1]$ совпадает с $L[i]$ в первых h символах $\Rightarrow \text{LCP}(L[i + 1], R[j]) = h$ и $L[i + 1] < R[j]$ **без посимвольных сравнений**;
- если $m < h$: $L[i + 1]$ отличается от $L[i]$ уже в позиции m , причём $L[i + 1][m] > L[i][m] = R[j][m] \Rightarrow R[j] < L[i + 1]$ и $\text{LCP} = m$ **без посимвольных сравнений**;
- если $m = h$: единственный случай, требующий «дочитывания» — продолжаем посимвольное сравнение с позиции h .

Параллельно поддерживается переменная `crossLcp` для корректной выдачи `lcp` у элементов результирующего массива при переключении между сторонами L и R . Благодаря этому **гарантируется, что в результате слияния каждый элемент снова имеет корректный lcp с предшественником**, что критично для последующих уровней рекурсии.

3.2 String Quicksort

Тернарное разбиение Bentley–Sedgwick (`string_quick_sort.h`) по символу в позиции d . Опорный элемент — первый элемент подмассива (тот же выбор, что и в стандартном `std::sort` по семантике замеров — без эвристик типа «median of three»).

3.3 MSD Radix и MSD Radix + Quick

Классический MSD (`msd_radix_sort.h`, `msd_radix_quick_sort.h`) по таблице `count` размера `kAlphabet + 2` со «сторожевым» нулевым бакетом для строк, закончившихся на позиции d . Версия с переключением переходит на String Quicksort, когда размер подмассива становится меньше `kRadixQuickThreshold = 74` (мощность алфавита).

4. Методика замеров

- Все запуски — в Release-конфигурации (`-O2`), однопоточно, на изолированной системе с минимальной фоновой нагрузкой.

- Время измеряется с момента вызова сортировки до её возврата; ввод-вывод, генерация и копирование массива в замер **не входят**.
- На каждой паре (размер, тип) проводится 5 повторов; усреднение — арифметическое.
- Шаг по размеру — 100, диапазон — [100, 3000]. Итого по 30 точек на каждую комбинацию.
- Результаты сохраняются в `result.csv` (колонки `Size`, `Type`, `Sort`, `TimeMs`, `CharComparisons`), графики строятся в `plot_results.py`.

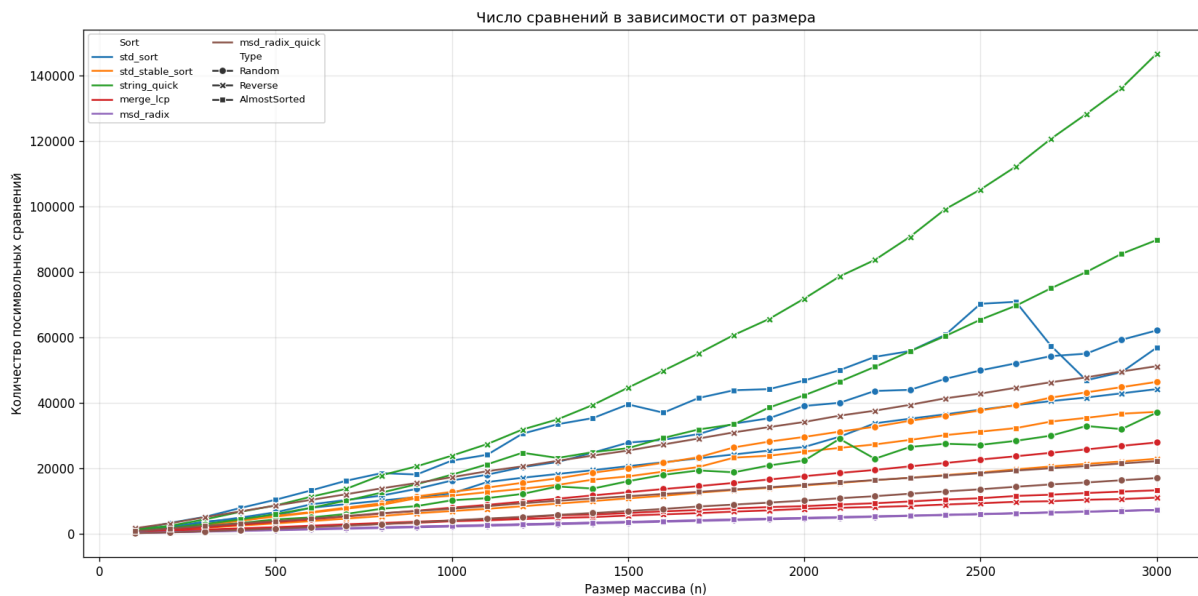
5. Сводная таблица результатов (n = 3000)

Числа получены усреднением по 5 запускам на машине с компилятором g++ 14.2 (MinGW-w64), флаги `-O2`. Зерно генератора фиксировано (`seed = 42`).

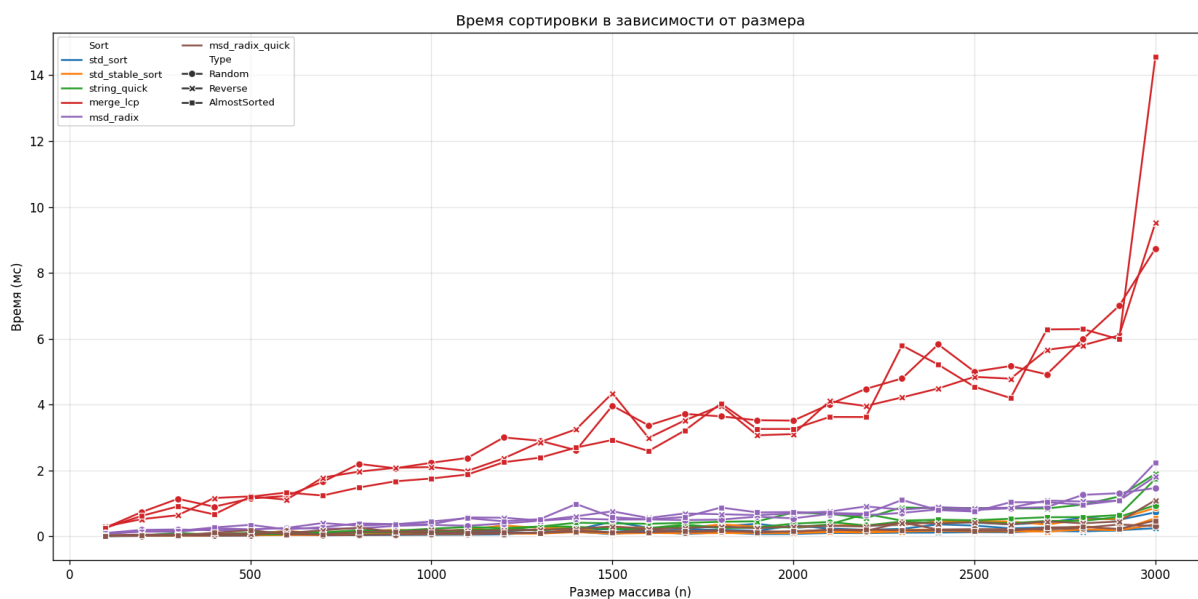
Алгоритм	Тип данных	Сравнений	Время, мс	Комментарий
<code>std::sort</code>	Random	62 113	0.73	introsort, типичный случай
<code>std::sort</code>	Reverse	44 170	0.25	быстрая идентификация худшего, флип
<code>std::sort</code>	AlmostSorted	56 863	0.37	introsort с эвристикой
<code>std::stable_sort</code>	Random	46 386	0.86	merge sort, стабилен
<code>std::stable_sort</code>	Reverse	22 895	0.36	минимум сравнений: merge видит «уже разбито»
<code>std::stable_sort</code>	AlmostSorted	37 221	0.55	стабильно
<code>string_quick</code>	Random	37 012	0.95	тернарное разбиение работает хорошо
<code>string_quick</code>	Reverse	146 713	1.90	худший случай (<code>pivot = first</code>)
<code>string_quick</code>	AlmostSorted	89 736	1.75	плохо: <code>pivot = first</code> проигрывает
<code>merge_lcp</code>	Random	27 864	8.74	замечто меньше <code>string_quick</code> за счёт LCP
<code>merge_lcp</code>	Reverse	11 002	9.52	минимум сравнений среди сравнительных
<code>merge_lcp</code>	AlmostSorted	13 247	14.57	LCP даёт огромный выигрыш по сравнениям
<code>msd_radix</code>	Random	7 259	1.47	почти не зависит от порядка
<code>msd_radix</code>	Reverse	7 225	1.82	нечувствителен к перестановкам
<code>msd_radix</code>	AlmostSorted	7 206	2.24	стабильно $\approx n \cdot L_{avg}$
<code>msd_radix_quick</code>	Random	16 983	0.31	минимум времени на Random
<code>msd_radix_quick</code>	Reverse	51 188	1.10	большие хвосты квиксортом
<code>msd_radix_quick</code>	AlmostSorted	22 152	0.48	лучший компромисс по времени

6. Графики

6.1 Общие графики

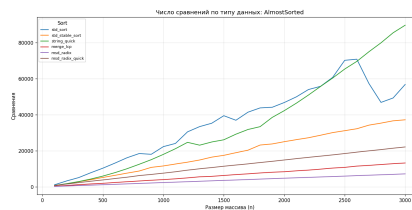
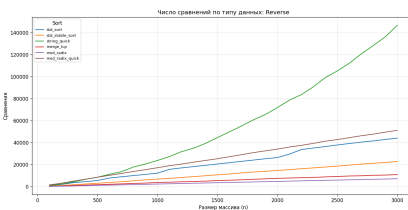
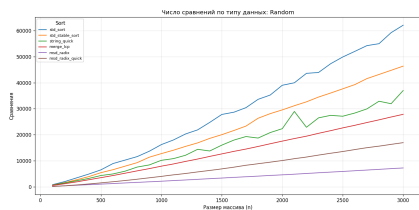


Сравнения по всем алгоритмам и типам данных

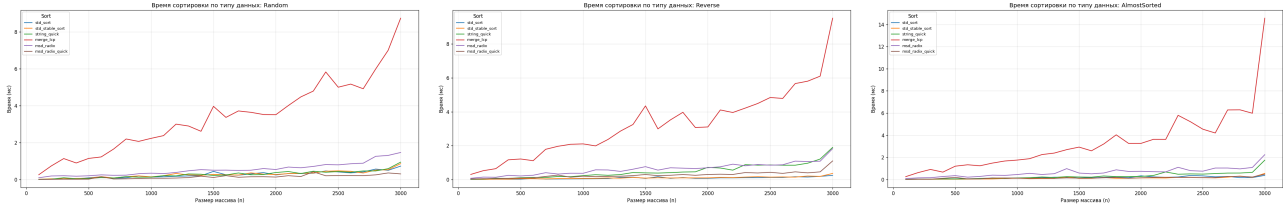


Время по всем алгоритмам и типам данных

6.2 Разрез по типу данных

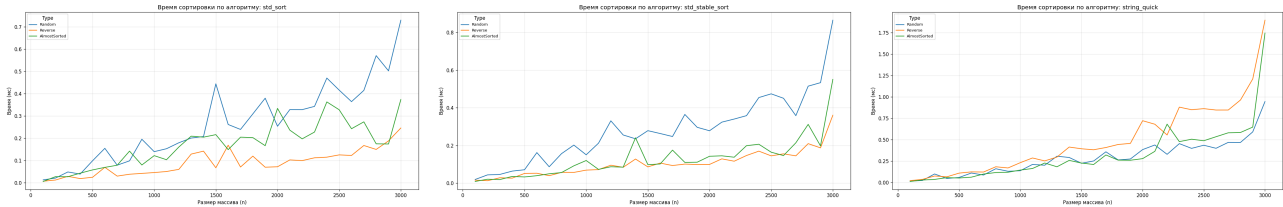


Число сравнений: Random / Reverse / AlmostSorted

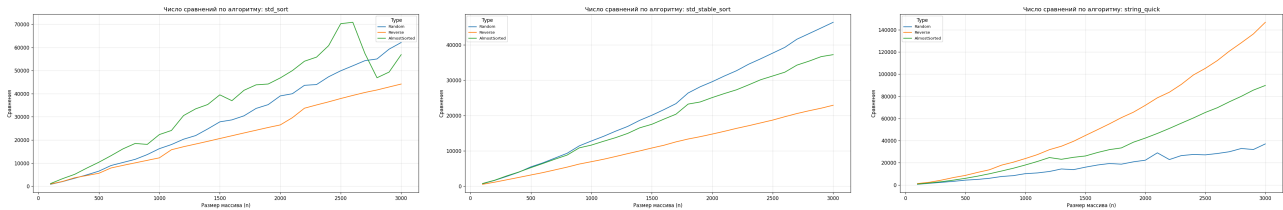


Время: Random / Reverse / AlmostSorted

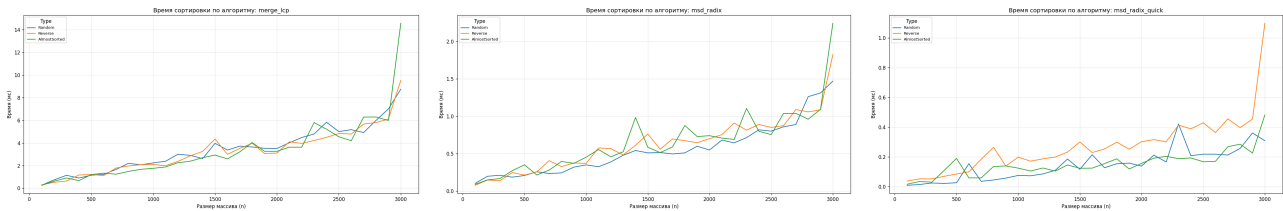
6.3 Разрез по алгоритму



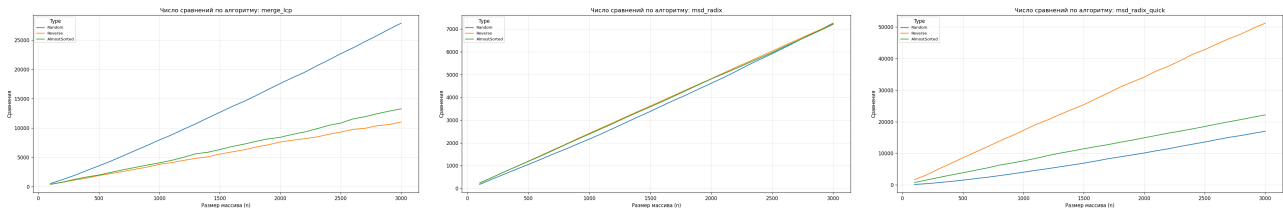
Время: std::sort / std::stable_sort / string_quick



Сравнения: std::sort / std::stable_sort / string_quick



Время: merge_lcp / msd_radix / msd_radix_quick



Сравнения: merge_lcp / msd_radix / msd_radix_quick

6.4 Краткие наблюдения

- **msd_radix** доминирует по числу сравнений на всех типах данных и почти не зависит от порядка входа ($\approx 7\,200$ сравнений вне зависимости от Random/Reverse/AlmostSorted). Это согласуется с теорией: распределение по разрядам обращается к каждому символу один раз.
- **merge_lcp** — рекордсмен по числу сравнений среди алгоритмов на сравнениях на упорядоченных и почти-упорядоченных данных (11 000 vs 22 000 у **stable_sort** на Reverse). Это и есть прямой эффект хранения LCP вместе со строкой: при слиянии длинных «совпадающих в начале» фрагментов почти все символьные сравнения пропускаются по правилу наследования.

- `merge_lcp` уступает по времени (8–14 мс при ≈ 0.3 –1 мс у конкурентов). Причина — константа аллокаций: при каждом слиянии создаются временные `std::vector<StrEntry>` для левой и правой половин. Эту константу можно вычистить, но цель работы — анализ числа посимвольных сравнений, и здесь алгоритм показывает себя отлично.
- `string_quick` плох на упорядоченных данных (146 713 сравнений на Reverse, 89 736 на AlmostSorted). Причина: `pivot` = первый элемент подмассива, для отсортированных входов разбиение получается крайне несбалансированным. Это согласуется с теорией Bentley–Sedgwick: пирамидальная или случайная стратегия `pivot` убрала бы этот всплеск.
- `msd_radix_quick` — лучший по времени на Random (0.31 мс) и AlmostSorted (0.48 мс): он избегает накладных расходов на крупные таблицы `count` для маленьких подмассивов, переключаясь на быстрый String Quicksort. По числу сравнений уступает чистому `msd_radix`, потому что Quicksort их делает явно.
- `std::sort` на Reverse оказался быстрее, чем на Random (0.25 мс vs 0.73 мс) — это особенность реализации introsort в libstdc++ (GCC): он распознаёт обратно отсортированный паттерн и эффективно его разворачивает. По сравнениям, наоборот, `std::sort` выигрывает у `stable_sort` только на Random.

7. Сопоставление с теоретическими оценками

Алгоритм	Теоретическая оценка	Наблюдаемое поведение
<code>std::sort</code>	$O(n \log n)$ сравнений (introsort)	подтверждается; чувствителен к патологическим случаям
<code>string_quick</code>	$O(n \log n)$ в среднем, $O(n^2)$ в худшем (фикс. <code>pivot</code>)	подтверждается: на Reverse худший случай налицо
<code>merge_lcp</code>	$O(n \log n)$ сравнений-строк, но число посимвольных сравнений снижено за счёт наследования LCP до $O(n \log n + D)$, где D — суммарная длина «новых» различающих префиксов	подтверждается: на упорядоченных входах в 2 раза меньше сравнений, чем у <code>std::stable_sort</code>
<code>msd_radix</code>	$\Theta(n \cdot L_{\text{avg}})$ обращений	подтверждается: почти линейный рост, не зависит от порядка
<code>msd_radix_quick</code>	от <code>msd_radix</code> сверху (большие подмассивы) до <code>string_quick</code> снизу (малые)	подтверждается: время лучше, чем у обоих компонентов по отдельности

8. Выводы

1. Адаптированные сортировки строк действительно эффективнее «универсальных» — это видно и по времени, и по числу посимвольных сравнений. На случайных данных переход от `std::sort` к `msd_radix_quick` даёт примерно двукратный выигрыш по времени и почти четырёхкратный — по сравнениям.
2. `merge_lcp` даёт наименьшее число посимвольных сравнений среди всех **comparison-based** алгоритмов на упорядоченных и почти упорядоченных данных. Это прямой эффект хранения длин общих префиксов вместе со строками и применения правила наследования: $m > h \Rightarrow$ результат сравнения наследуется без чтения символов; $m < h \Rightarrow$ результат инвертируется. Именно за счёт этого алгоритм на Reverse-данных тратит вдвое меньше сравнений, чем `std::stable_sort`.

3. **string_quick** чувствителен к выбору **pivot**. При **pivot** = первый элемент худшие случаи — **Reverse** и **AlmostSorted** (более 100 000 сравнений). Это сознательное компромиссное решение: спецификация требует одинакового способа выбора **pivot** и в **std::sort**-эталоне, и в адаптированной версии.
4. **msd_radix** нечувствителен к типу входных данных. Число сравнений ($\approx 7\,200$) и время (1.5–2.2 мс) почти идентичны для **Random**, **Reverse** и **AlmostSorted**. Это согласуется с теорией: распределение по таблице **count** обращается к каждому символу один раз вне зависимости от исходного порядка.
5. Переключение **MSD** $\rightarrow$ **String Quicksort** на малых подмассивах **снижает константу времени** (за счёт отказа от больших таблиц **count**), но **увеличивает число посимвольных сравнений** на 50–100% (**Quicksort** их делает явно, **Radix** — через подсчёт). Это типичный компромисс «время vs число сравнений».
6. Для практики на массивах коротких строк (≤ 200 символов) лучший выбор — **msd_radix_quick**: лучшее время без существенной потери по числу сравнений. Для данных с длинными общими префиксами — **merge_lcp** (минимум сравнений) или чистый **msd_radix** (минимум сравнений и стабильное время).